

Catalogizer

Erweiterte Verwaltung von Medienkollektionen mit Multi-Protokoll-Unterstützung - alles, was Sie besitzen, erkennen, katalogisieren und anreichern.

Zusammenfassung

Catalogizer ist ein selbstgehostetes Medienverwaltungssystem, das Medien automatisch erkennt, kategorisiert und organisiert – über SMB, FTP, NFS, WebDAV und lokale Dateisysteme hinweg. Es bietet Echtzeit-Überwachung, verschlüsselte Speicherung, externe Metadatenanreicherung sowie eine moderne React-Benutzeroberfläche, gestützt auf eine leistungsstarke Go-API-Architektur.

Kurzbeschreibung

Ein produktionsreifes Medienbibliotheks-Management mit Multi-Protokoll-Unterstützung. Ein Go/Gin-REST-API erkennt über 50 Medientypen aus SMB/FTP/NFS/WebDAV/lokalen Quellen, reichert sie mit Daten aus TMDB/IMDb/MusicBrainz/Steam und weiteren an und stellt eine Echtzeit-React-Webanwendung über eine verschlüsselte SQLCipher-Datenbank bereit.

Ausführliche Beschreibung

Die meisten Medienmanager verlangen von Ihnen, sich zuerst unterzuordnen: Alles muss auf einer Festplatte, in einem Format und von einem Typ sein – erst dann helfen sie weiter. Catalogizer geht von der gegenteiligen Prämisse aus – Ihre Sammlung existiert bereits dort, wo sie existiert, verteilt auf NAS-Freigaben und Protokolle, die sich nie einigen werden – und trifft sie genau dort an. Es beherrscht die Protokolle, die Speichersysteme bereits nutzen – SMB/CIFS, FTP/FTPS, NFS, WebDAV und das lokale Dateisystem – hinter einer einheitlichen Client-Abstraktion, sodass eine Windows-Freigabe, ein FTP-Archiv und ein WebDAV-Mount für die darüberliegenden Schichten identisch erscheinen und ohne Änderungen am Anwendungscode gemischt, ausgetauscht

oder stillgelegt werden können. Ein Go-Backend (Gin-REST-API) überwacht diese Quellen kontinuierlich, erkennt und klassifiziert über 50 Medientypen (Filme, Serien, Musik, Spiele, Software, Dokumentationen), sobald Dateien auftauchen, und reichert jeden Eintrag mit Daten aus einer Reihe externer Anbieter an – TMDb, IMDb, TVDB, MusicBrainz, Spotify, Steam und mehr. So wird aus einem bloßen Dateinamen ein vollständig attributierter Katalogeintrag mit Coverbildern, Besetzung und Metadaten. Die Ergebnisse werden über WebSockets an eine TypeScript-React-Oberfläche gestreamt, sodass sich die Bibliothek live aktualisiert, während die Erfassung läuft, und nicht erst nach einem manuellen Refresh. Jedes Byte der Metadaten wird in einer verschlüsselten SQLCipher-Datenbank gespeichert, die durch JWT-basierte, rollenbasierte Authentifizierung geschützt ist.

Während die meisten Katalogisierer bei einem Ausfall einer Freigabe stillschweigend versagen, ist Catalogizer darauf ausgelegt, auch bei Störungen nützlich zu bleiben. Ein temporärer SMB-Ausfall wird durch exponentielles Backoff bei der Wiederverbindung, einen Circuit-Breaker, der verhindert, dass ein ausgefallener Host ständig angefragt wird, kontinuierliche Gesundheitsüberwachung und einen Offline-Metadaten-cache abgefedert, der Nutzeranfragen mit dem letzten bekannten Zustand beantwortet – der Unterschied zwischen *„Die gesamte Anwendung ist offline, weil ein NAS neu gestartet wurde“* und *„Eine Quelle ist beeinträchtigt, aber alles andere funktioniert“*. Neben der Katalogisierung dient es auch als operatives Werkzeug für die Sammlung: Analysen zu Wachstumstrends und Qualitäts-/Versionsverfolgung, professionelle PDF-Berichtserstellung, ein PDF-zu-Bild/Text/HTML-Konvertierungsdienst, Export/Import von Favoriten (JSON/CSV) sowie Cloud-Synchronisation mit S3, Google Cloud Storage oder lokalen Ordnern. Und es ist kein monolithisches System, das zufällig groß ist – es besteht bewusst aus 21 wiederverwendbaren `digital.vasic.*-Go`-Submodulen sowie TypeScript-Client-Paketen, die jeweils unabhängig getestet und versioniert sind. Dieselben erprobten Authentifizierungs-, Dateisystem-, Streaming- und Überwachungskomponenten, die Catalogizer antreiben, kommen auch in der gesamten Produktfamilie zum Einsatz. Die Qualitätssicherung erfolgt nicht durch Eigenangaben: Das Challenges-Framework und HelixQA unterziehen jede beworbene Funktion einer nachweisgestützten Überprüfung ohne Bluff.

Warum wir es entwickelt haben

Bestehende Medienmanager gehen von einem einzigen Speicher-Backend und einem einzigen Medientyp aus. Echte Sammlungen verteilen sich jedoch über zahlreiche NAS-Freigaben und Protokolle, leiden unter Ausfällen einzelner Freigaben und bestehen aus einer Mischung von Filmen, Musik, Spielen und

Software. Catalogizer wurde entwickelt, um alle Protokolle gleichwertig zu behandeln, instabile Netzwerkspeicher zu überstehen und einen einzigen, angereicherten, verschlüsselten Katalog über alle Inhalte hinweg bereitzustellen.

Warum es ein Game-Changer ist

Es vereint in einem selbstgehosteten, verschlüsselten Paket, wofür normalerweise ein ganzer Stapel separater Tools nötig wäre: protokollunabhängige Erfassung, die jedes Speicher-Backend gleich behandelt, Resilienz, die den Katalog auch bei Speicherausfällen verfügbar hält – statt mit ihnen abzustürzen – und eine umfassende Anreicherung durch mehrere Anbieter, die aus Rohdateien eine durchsuchbare, attributierte Bibliothek macht. Der Vorteil der modularen Architektur liegt in ihrer Skalierbarkeit: Eine Verbesserung des Dateisystem-Clients oder ein neues Provider-Plugin muss nur einmal implementiert werden und kommt allen Nutzern zugute. So wird Catalogizer kontinuierlich besser, während sich das Ökosystem um es herum weiterentwickelt. Kurz gesagt: Es ist der Unterschied zwischen einem Medienindex und einem Mediensystem – eines, das Ihnen gehört, das instabile Infrastruktur übersteht und dessen Funktionsweise bewiesen statt nur versprochen ist.

Was innovativ ist

- Einheitlicher Multi-Protokoll-Dateisystem-Client (SMB/FTP/NFS/WebDAV/lokal) hinter einer einzigen Schnittstelle.
- Offline-Cache mit Circuit Breaker, damit der Katalog auch bei Speicherausfällen nutzbar bleibt.
- Vollständige Aufteilung in 21 wiederverwendbare `digital.vasic.*-Go-Submodule` und `TS-Client-Module`.
- Verschlüsselter Katalog im Ruhezustand (SQLCipher) mit Echtzeit-WebSocket-Synchronisation zur Benutzeroberfläche.
- Evidenzbasierte Qualitätssicherung über das Challenges-Framework und HelixQA-Integration.

Herausforderungen & Lösungen

- **Instabile Netzwerkspeicher:** Gelöst durch exponentielles Backoff, Circuit Breaker, Health Checks und einen Offline-Cache mit Verdrängungsrichtlinie, der zwischengespeicherte Metadaten bereitstellt, wenn Quellen nicht erreichbar sind.
- **Protokollvielfalt:** Gelöst durch die Abstraktion aller Protokolle hinter einem gemeinsamen `digital.vasic.filesystem-Client`, sodass höhere Ebenen protokollunabhängig arbeiten.

- **Datensicherheit:** Gelöst durch SQLCipher-Verschlüsselung im Ruhezustand sowie JWT/RBAC-Authentifizierung und Middleware zur Anfragesanitisierung.
- **Wartbarkeit im großen Maßstab:** Gelöst durch die Auslagerung aller generischen Logik in unabhängig getestete Submodule statt einer monolithischen Architektur.

Technologie-Stack (Warum + Wie)

- **Go + Gin** - Hochleistungs-REST-API-Kern (catalog-api); gewählt für Nebenläufigkeit und Durchsatz bei kontinuierlichen Überwachungsaufgaben.
- **TypeScript + React + Tailwind (Vite)** - Reaktionsschnelle catalog-web-Benutzeroberfläche mit Echtzeit-Updates.
- **WebSockets** - Live-Datensynchronisation zwischen Backend-Hub und Benutzeroberfläche.
- **SQLCipher (verschlüsselter SQLite)** - Verschlüsselter Metadatenpeicher im Ruhezustand; duale SQLite/PostgreSQL-Unterstützung über `digital.vasic.database`.
- **SMB/FTP/NFS/WebDAV-Clients** - Multi-Protokoll-Erfassung über `digital.vasic.filesystem`.
- **Externe Metadaten-APIs (TMDB, IMDB, TVDB, MusicBrainz, Spotify, Steam)** - Provider-Plugins zur Anreicherung.
- **Prometheus + OpenTelemetry** - Metriken/Tracing über `digital.vasic.observability`.
- **Docker / Builder-Container** - Reproduzierbare Builds (Tauri/Rust über catalogizer-builder geroutet).
- **Redis** - Caching/Rate Limiting über `digital.vasic.cache / ratelimiter`.
- **S3 / Google Cloud Storage** - Cloud-Synchronisation und Checkpoint-Speicherung.